Offline Multi-Pedestrian Tracking on MOT16: Two Pipelines Compared

Tony Nguyen Rehan Ali Bosia N'dri
University of Missouri–Kansas City
COMP-SCI 5567 Deep Learning

Fall 2026

Abstract

Multi-object tracking finds every person in a video and keeps one identity for each person over time. We build and compare two offline tracking-by-detection pipelines on the MOT16 benchmark. Pipeline A uses a Faster R-CNN detector, a Siamese ReID network, and a custom tracker. Pipeline B uses a YOLO detector, a larger ReID network, the BoT-SORT tracker, and an appearance-based split and merge repair step. Both pipelines train on six sequences and are scored with TrackEval on the held-out sequence MOT16-09. We report three findings. First, the final outputs cover a similar share of pedestrians (recall 0.769 for Pipeline A and 0.779 for Pipeline B), but Pipeline B outputs fewer false boxes (462 against 1,073). Second, Pipeline A keeps identities more stable than BoT-SORT alone (35 against 46 ID switches; AssA 47.11 against 38.95), and the repair raises Pipeline B from HOTA 48.40 to 55.30 and from IDF1 55.65 to 68.04, but also from 46 to 56 ID switches. Third, no run tracks more than 2 of 18 long-lived people with one ID, and heavy occlusion is the main limit. All thresholds and checkpoints were chosen on the same sequence as the scores, so the scores are optimistic.

1 Introduction

Tracking people in video supports many systems, for example autonomous driving, surveillance, sports analytics, robotics, and retail analytics. A detector answers where the people are in each frame. A tracker must also answer who is who, so that each person keeps one identity over time.

This second question is hard. In crowded scenes, people hide each other and often wear similar clothes. A person can disappear behind a pole for a few frames and return with a new identity, or a nearby person can take over the old identity. We saw both cases in our test videos. Such errors can lower identity-based scores even when the detector is good.

We study this problem on MOT16 [12], a benchmark of crowded street videos. We follow the tracking-by-detection approach: a detector finds people in each frame, and a tracker links the boxes over time, as in SORT, DeepSORT, ByteTrack, and BoT-SORT [3, 18, 19, 1]. We process whole videos, so our methods are offline and can use steps that an online tracker cannot use. We build two pipelines. Pipeline A follows the course template: Faster R-CNN [15], a Siamese ReID network, and a custom tracker. Pipeline B uses a YOLO detector [14], a larger ReID network, the BoT-SORT tracker, and our own appearance-based repair step.

At first, each pipeline reported its own metrics with its own protocol, so the numbers could not be compared. Our contributions are:

- two complete pipelines that use the same data split and are evaluated with the same code (TrackEval, which implements HOTA [10]);

- a common evaluation: the metric code of Pipeline A checked against TrackEval, one detector evaluator, and one ReID protocol;
- controlled experiments: an ablation of Pipeline A, a repair study for Pipeline B, a detector swap inside the tracker of Pipeline A, and a per-person analysis;
- an honest account of failures, including target videos on MOT16 test sequences.

Section 2 describes the setup and both pipelines, Section 3 lists the challenges, Section 4 gives the results and self-evaluation, and Section 5 concludes.

2 Methodology

2.1 Shared setup

Data. MOT16 has seven training sequences with labels and seven test sequences without public labels. We split by sequence. Both pipelines train on MOT16-02, 04, 05, 10, 11, and 13, and we validate on MOT16-09 (525 frames, 1920×1080, 25 identities, 5,257 evaluated pedestrian boxes). A split by sequence avoids near-identical neighbouring frames in the validation set. The test sequences are used only for qualitative target videos.

Annotation rules. We evaluate pedestrians (class 1) that the benchmark marks for evaluation. Persons on vehicles, static persons, distractors, reflections, and pedestrians with evaluation flag 0 are "don't care" regions: a prediction on them is not an error.

Metrics. We score tracks with TrackEval (HOTA, CLEAR, Identity) on MOT16-09 and report HOTA [10] with its parts DetA and AssA, MOTA [2], IDF1 [16], ID switches (IDSW), false positives (FP), and false negatives (FN). Pipeline A computed MOTA and IDF1 with its own code. On its saved tracks, TrackEval gives the same values (MOTA 55.83 against 55.8, IDF1 60.35 against 60.3, 35 ID switches, 1,073 false positives, and 1,214 misses), so its own code is reliable on this sequence. For detectors, we use one evaluator for both: all 525 frames, IoU 0.5, one-to-one matching in score order, and detections on "don't care" regions are ignored. We also report recall by box visibility at equal precision 0.80. For ReID, we report mAP and Rank-k. Each pipeline first used its own protocol. Pipeline A used every crop as a query and removed same-person crops within 15 frames (24 unseen people, 747 crops). Pipeline B used early crops as queries and late crops as the gallery, with a 20% gap. We then apply the protocol of Pipeline B to all four models (two trained models and two ImageNet baselines) and call it the common protocol.

Tuning. Both pipelines chose thresholds or checkpoints on MOT16-09, so all scores are optimistic.

2.2 Pipeline A

Detector. We use the torchvision Faster R-CNN [15] with a ResNet-50 [4] and FPN [9] backbone (v2 weights), COCO weights, and a new two-class head (background and pedestrian). The backbone is frozen, so 16.4 M of 43.3 M parameters are trained. Training uses 4 epochs, SGD with learning rate 0.005, batch size 4, every second frame of the training sequences, and flip, colour, and blur augmentation.

ReID. A Siamese network with shared weights [17] embeds each crop. It is a ResNet-18 with a 128-dimensional embedding, trained with batch-hard triplet loss [5] (margin 0.3) on 20,146 crops of 362 people (every third frame, visibility ≥ 0.5, height ≥ 50 px). Training runs for 25 epochs of 200 batches with Adam (learning rate $3 \cdot 10^{-4}$) and random erasing [20].

Tracker. Each track has a Kalman filter [7]. Detections and tracks are matched with the Hungarian algorithm [8] on the cost $0.6\, d_{\text{app}} + 0.4\,(1 - \text{IoU})$, where d_{app} is the appearance distance. A distance gate removes pairs that are too far apart. The first pass uses detections with score ≥ 0.7. The second pass uses scores from 0.6 up to 0.7 and matches by IoU, in the spirit of ByteTrack [19]. A track is confirmed after 3 hits and is deleted after 60 missed frames. The final appearance threshold is 0.15. Three offline rules follow: a track is split when the person leaves the picture, gaps of up to 5 frames are filled by interpolation, and tracks shorter than 8 frames are dropped.

2.3 Pipeline B

Detector. We fine-tune YOLO [14] (model yolo26m, Ultralytics software [6]) with all layers, image size 960, up to 30 epochs, a cosine schedule, early stopping, and the standard Ultralytics augmentation.

ReID. The ReID network is a ResNet-50 with last stride 1 and global pooling, a BNNeck [11], and a 2048-dimensional embedding. The loss is identity cross-entropy with label smoothing 0.1 plus batch-hard triplet loss (margin 0.3). Training runs for 12 epochs with random erasing, on crops from every fifth frame with visibility ≥ 0.4. We keep the checkpoint with the best validation mAP.

Tracker. We use BoT-SORT [1] from the Ultralytics library, with a Kalman filter and camera-motion compensation. ReID is not used inside this tracker.

Repair step (our own code). After tracking, we compute an embedding for every box. A track is split where the mean embeddings of the previous 8 and the next 8 boxes differ by a cosine distance above 0.5. Then non-overlapping tracks are merged when the cosine distance of their mean embeddings is below 0.35, the time gap is at most 150 frames, and the position after the gap is plausible.

Detector swap. To separate detector and tracker effects, we feed the YOLO detections, embedded by the ReID model of Pipeline A, into the tracker of Pipeline A. We test four pairs of score thresholds: 0.7/0.6, 0.6/0.5, 0.5/0.4, and 0.4/0.3.

2.4 Implementation details

Pipeline A runs on a free Colab T4 GPU. Pipeline B runs on an NVIDIA A100-SXM4-40GB GPU; its detector training took about 5,476 s for 30 epochs, and the best epoch was 21. All code uses PyTorch [13], and checkpoints are saved to Google Drive with automatic resume after a Colab disconnect. In Pipeline A, the detector comes from torchvision, and the tracker, the ReID training, and the metric code are our own. In Pipeline B, the detector and BoT-SORT come from the Ultralytics library, and the ReID training, the repair step, the evaluation, and the target selection are our own. Table 1 summarizes both pipelines.

Table 1: Design of the two pipelines.

	Pipeline A	Pipeline B
Detector	Faster R-CNN, ResNet-50-FPN v2, backbone frozen, 4 epochs	YOLO (yolo26m), all layers, up to 30 epochs
ReID network	Siamese ResNet-18, 128-d	ResNet-50 with BNNeck, 2048-d
ReID loss	batch-hard triplet	identity cross-entropy + batch-hard triplet
ReID crops	every 3rd frame, visibility ≥ 0.5	every 5th frame, visibility ≥ 0.4
Tracker	own: Kalman filter, Hungarian matching, gating, two passes	BoT-SORT (library)
Offline steps	exit rule, gap fill (5 frames), drop tracks under 8 frames	split and merge repair with ReID
Metric code	own code, checked against TrackEval	TrackEval
GPU	Colab T4	A100-SXM4-40GB

3 Challenges and How We Solved Them

Identity switches and fragmentation after short occlusions. In MOT16-03, a man who crossed behind a light pole received two IDs (235 and 263), and the track of the target with ID 166 changed its ID near frame 735. Pipeline A puts appearance into the matching step, and it has fewer ID switches than BoT-SORT alone (35 against 46) and a higher AssA (47.11 against 38.95). Pipeline B repairs tracks offline. The repair raises HOTA from 48.40 to 55.30 and IDF1 from 55.65 to 68.04, but it also raises the ID switches from 46 to 56. The problem is not solved: Section 4.4 shows that most people still receive several IDs.

Heavy occlusion. Most missed pedestrians of Pipeline B have visibility below 0.25 (79% of its misses). All pedestrians of MOT16-09 are at least 120 px tall, so small size is not the main limit on this sequence. In the lowest visibility bin, the detectors reach recall 0.471 (Pipeline B) and 0.316 (Pipeline A) at precision 0.80, and the final outputs reach 0.420 and 0.348. We did not add a method for this problem. Gap filling of up to 5 frames in Pipeline A cannot cover long occlusions.

Optimistic validation. Early prototypes used random frame splits and ReID validation on identities that were also in training, which gave numbers that were too good. We changed to a split by sequence, and we test ReID on unseen identities. A limit remains: both pipelines still tuned thresholds or checkpoints on MOT16-09 (Section 4.6).

Metrics that cannot be compared. Each pipeline first used its own metric code and its own ReID protocol. We scored all tracks with TrackEval, used one detector evaluator, and used one ReID protocol for all models. The scores of YOLO and Faster R-CNN have different scales, so one threshold cannot be used for both. We therefore tested four threshold pairs in the detector swap. The best pair was chosen on the same sequence, so that result is optimistic.

People who are not evaluated. The benchmark ignores static people, reflections, and distractors. In MOT16-03, Pipeline B did not detect people who were sitting or standing still. A likely reason is that static persons were not in our training labels; we did not test this. Two false tracks (IDs 144 and 154) also appeared near a lit entrance cover, and we do not know the cause.

Limited compute. Colab sessions disconnect, and we found unreadable image frames after copying a sequence to the local disk. We re-copied the missing frames from Google Drive, and we saved checkpoints to Drive with automatic resume.

Library code and own code. The tracker of Pipeline B comes from a library, so it is not our own work. Our own parts in Pipeline B are the ReID training, the repair step, the evaluation, and the target selection. Pipeline A has its own tracker, and the detector swap puts the YOLO detections into that tracker to compare the two detectors under one tracker.

4 Results and Self-Evaluation

All results are on the held-out sequence MOT16-09 unless we say otherwise. Test sequences have no public labels, so their results are qualitative.

4.1 Detection

Table 2 shows the evaluation that each pipeline made with its own code. The two columns are not directly comparable: they use different AP code, and Pipeline A appears to use every third frame of MOT16-09 (its notebook prints 175 frames). The Ultralytics evaluation of Pipeline B counts detections on non-evaluated people as false positives.

Table 2: Detector results from each pipeline's own evaluation on MOT16-09.

Pipeline	Model	AP@0.5 / mAP@0.5	Precision	Recall
A	Faster R-CNN, COCO weights	0.741	0.664	0.751
A	Faster R-CNN, fine-tuned	0.780	0.743	0.769
B	YOLO, pretrained	0.792	0.904	0.728
B	YOLO, fine-tuned	0.872	0.905	0.780

We then evaluate both detectors with one evaluator (Section 2.1). Figure 1 shows the precision–recall curves. Both detectors keep precision near 0.98 to 1.0 until recall of about 0.7, and Pipeline A is slightly higher there. The curves cross near recall 0.7. Above that point, Pipeline B keeps higher precision, and its maximum recall is about 0.82 against about 0.81 for Pipeline A (read from the figure). Table 3 gives the AP and the precision and recall at three score thresholds. The AP of Pipeline B (0.7896) is close to the AP of Pipeline A (0.7798). The AP of Pipeline A under this evaluator is close to the value in its own notebook (0.780), so the evaluator agrees with its code. The score scales of the two detectors differ, so equal thresholds in this table do not mean equal behavior.

Table 3: AP@0.5 and precision (P) and recall (R) at three score thresholds, same evaluator (MOT16-09).

Detector	AP@0.5	P@0.25	R@0.25	P@0.5	R@0.5	P@0.7	R@0.7
Pipeline B, YOLO	0.7896	0.868	0.775	0.928	0.729	0.962	0.678
Pipeline A, Faster R-CNN	0.7798	0.644	0.783	0.744	0.770	0.795	0.758

Table 4 compares recall by box visibility at the same precision (0.80). Pipeline B finds more heavily hidden people (0.471 against 0.316 at visibility below 0.25) and has a higher overall recall

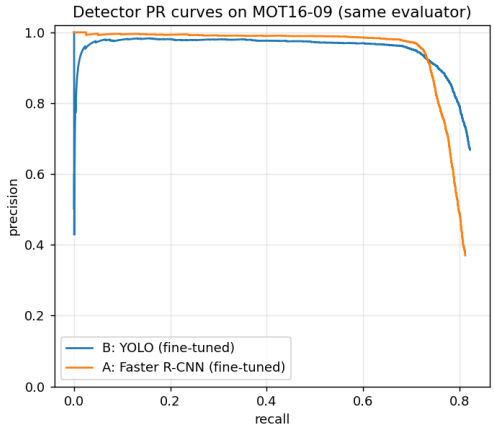


Figure 1: Precision–recall curves of both detectors on MOT16-09 with the same evaluator.

(0.798 against 0.757). For visibility from 0.5 upwards, the two detectors are close, and Pipeline A is slightly higher. These results hold for this precision and this sequence only.

Table 4: Recall by box visibility at precision 0.80 (same evaluator, MOT16-09).

Detector (score threshold)	0–0.25	0.25–0.5	0.5–0.75	0.75–1	All
Boxes in bin	1,564	577	612	2,504	5,257
Pipeline B, YOLO (≥ 0.13)	0.471	0.787	0.912	0.977	0.798
Pipeline A, Faster R-CNN (≥ 0.72)	0.316	0.797	0.915	0.984	0.757

Table 5 shows the same matching rule on the final track outputs. The recall is similar (0.769 and 0.779), and Pipeline A outputs more unmatched boxes (1,073 against 478). TrackEval counts 462 false positives and 1,146 misses for Pipeline B because it handles ignore regions slightly differently; the values for Pipeline A are identical.

Table 5: Coverage of the final track outputs (our matching rule, MOT16-09).

Output	Recall	Missed	Unmatched boxes	Precision
Pipeline A (published tracks)	0.769	1,214	1,073	0.790
Pipeline B (BoT-SORT)	0.779	1,162	478	0.895

4.2 Re-identification

Table 6 gives the result of each pipeline under its own protocol. In both, the trained model is better than the ImageNet baseline. The two protocols differ, so we do not compare the rows across pipelines.

Table 7 applies one protocol to all four models. Under this protocol, the trained model of Pipeline B scores higher than the trained model of Pipeline A (mAP 0.844 against 0.635; Rank-1 0.922 against 0.767). The Rank-1 of Pipeline A falls from 0.914 under its own protocol to 0.767 under the common protocol. The comparison has three caveats. Pipeline B uses a larger network and a larger input, so size and training method are mixed. Its checkpoint was chosen on this validation sequence (epoch 5; later epochs scored about 0.79 to 0.82 mAP). The test crops come from the crop extraction of Pipeline B. Figure 2 shows retrieval examples of Pipeline B.

Table 6: ReID results under each pipeline's own protocol (MOT16-09).

Model	Rank-1	mAP	Rank-5
A: ResNet-18, ImageNet only	0.812	0.490	–
A: Siamese ResNet-18, triplet loss	0.914	0.728	–
B: ResNet-50, ImageNet only	0.533	0.460	–
B: ResNet-50 + BNNeck, ID + triplet loss	0.922	0.844	0.942

Table 7: ReID results under the common protocol (MOT16-09).

Model	mAP	Rank-1	Rank-5	Rank-10
A: ResNet-18, ImageNet only	0.400	0.564	0.759	0.860
A: Siamese ResNet-18, triplet loss	0.635	0.767	0.872	0.879
B: ResNet-50, ImageNet only	0.460	0.533	0.786	0.860
B: ResNet-50 + BNNeck, ID + triplet loss	0.844	0.922	0.942	0.946

4.3 Tracking

Table 8 lists all runs, scored by TrackEval. Figure 3 shows HOTA, IDF1, and MOTA for the saved tracks.

Table 8: TrackEval results on MOT16-09. Rows marked "hybrid" use YOLO detections in the tracker of Pipeline A.

Run	HOTA	DetA	AssA	MOTA	IDF1	IDSW	FP	FN	IDs
A published tracks	50.40	53.97	47.11	55.83	60.35	35	1073	1214	54
A reproduced here	50.21	53.94	46.79	55.85	62.38	33	1073	1215	54
B: pretrained YOLO + ByteTrack	48.11	54.50	42.71	59.29	57.37	46	735	1359	92
B: fine-tuned YOLO + ByteTrack	47.94	58.99	39.12	64.77	55.77	42	532	1278	73
B: fine-tuned YOLO + BoT-SORT	48.40	60.36	38.95	68.54	55.65	46	462	1146	72
B: BoT-SORT + repair (0.5 / 0.35)	55.30	60.25	50.89	68.35	68.04	56	462	1146	72
Hybrid, thresholds 0.7 / 0.6	50.12	56.41	44.57	66.24	61.88	21	145	1609	31
Hybrid, thresholds 0.6 / 0.5	51.08	58.18	44.93	67.38	62.65	30	229	1456	39
Hybrid, thresholds 0.5 / 0.4	47.73	59.56	38.37	68.61	54.70	40	290	1320	41
Hybrid, thresholds 0.4 / 0.3	47.91	59.96	38.40	69.15	55.93	50	364	1208	49

Pipeline A against BoT-SORT. Pipeline A has fewer ID switches (35 against 46), fewer IDs (54 against 72), and a higher AssA (47.11 against 38.95) than the BoT-SORT run of Pipeline B. Pipeline B has a higher DetA (60.36 against 53.97) and a higher MOTA (68.54 against 55.83), mainly because it has fewer false positives (462 against 1,073). Appearance in the matching step, the offline rules, and strict score thresholds are likely reasons for the stability of Pipeline A; we did not test each one. The ablation in Table 9 is consistent with this: the Kalman filter step has an IDF1 (56.2) close to the BoT-SORT run (55.65), and the Siamese step raises it to 62.2. The last two steps lower IDF1 (60.2 and 60.3) and MOTA (56.0 and 55.8), raise the false positives (998 and 1,073), and lower the misses (1,275 and 1,214). A likely reason is that filled gaps add boxes where no pedestrian is annotated; we did not test this.

Repair step of Pipeline B. The repair raises HOTA from 48.40 to 55.30, AssA from 38.95 to 50.89, and IDF1 from 55.65 to 68.04, and it does not change DetA or MOTA much (60.36 to 60.25 and 68.54 to 68.35). The ID switches rise from 46 to 56. Merging alone (threshold 0.35, no split)

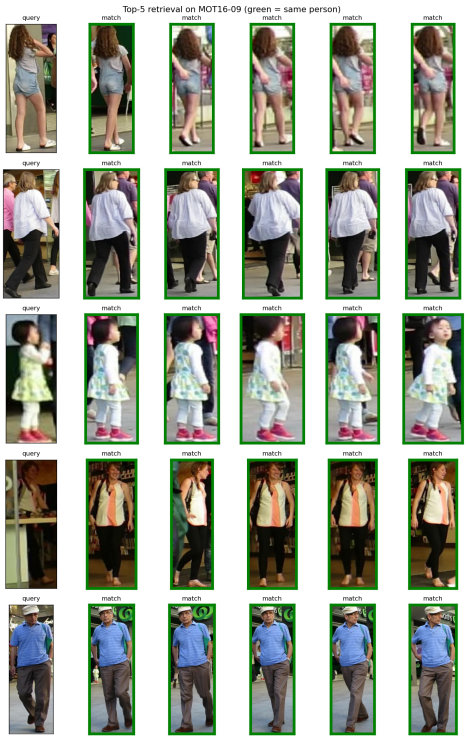


Figure 2: Top-5 retrieval examples of the ReID model of Pipeline B on MOT16-09 (green border: same person).

Table 9: Ablation of the tracker of Pipeline A on MOT16-09 (cumulative steps).

Configuration	MOTA	IDF1	IDSW	FP	FN	IDs
IoU only	56.3	47.7	71	930	1295	42
+ Kalman filter	57.6	56.2	43	895	1290	37
+ Siamese ReID	57.4	62.2	34	919	1288	43
+ low-score second pass	56.0	60.2	38	998	1275	47
+ exit rule and gap fill (final)	55.8	60.3	35	1073	1214	49

gives HOTA 52.03, AssA 45.03, IDF1 62.58, MOTA 68.61, 42 ID switches, and 58 IDs. A split threshold of 0.4 cuts too often (74 to 75 ID switches). A likely reason for the extra ID switches is that each cut can count as a switch; we did not test this.

Detector swap. At thresholds 0.6/0.5, YOLO detections in the tracker of Pipeline A give 229 false positives against 1,073, and 1,456 misses against 1,215 for its own detections. MOTA rises from 55.85 to 67.38, HOTA from 50.21 to 51.08, and IDF1 from 62.38 to 62.65, and the ID switches fall from 33 to 30. Lower thresholds reduce the misses (1,208 at 0.4/0.3) but also reduce AssA (38.40 at 0.4/0.3), and the lowest ID switches (21) occur at 0.7/0.6. The best pair was chosen on this sequence, so these values are optimistic.

4.4 Per-person analysis and target videos

Table 10 counts, for people who appear in at least 100 frames (18 of 25 people), how many are followed by one ID. A person is "perfect" if one predicted ID covers at least 90% of the frames.

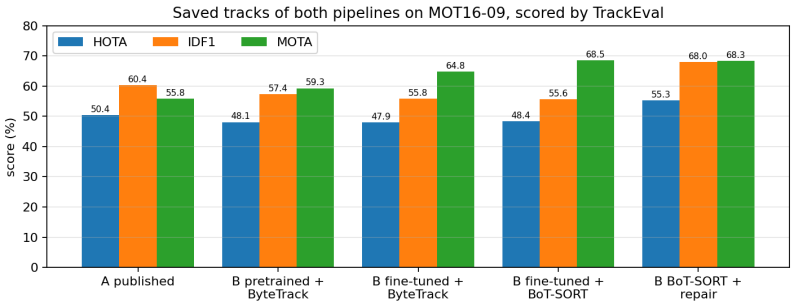


Figure 3: HOTA, IDF1, and MOTA of the saved tracks of both pipelines on MOT16-09 (TrackEval).

The row of Pipeline A is identical to the file of Pipeline A itself. No run has more than 2 perfect people, and each person receives between 2.78 and 3.78 IDs on average. The repair raises the IDs per person (3.50 to 3.78) while it raises IDF1 and AssA, so its gain probably comes from cutting segments with a wrong identity and not from joining fragments; we did not test this. The hybrid has the fewest IDs per person (2.78) but lower coverage (0.720).

Table 10: Per-person results for the 18 long-lived people of MOT16-09 (same matching rule for all runs).

Run	One ID	Perfect	Mean coverage	IDs per person
A published	5	2	0.767	2.94
B: BoT-SORT	3	1	0.775	3.50
B: BoT-SORT + repair	3	1	0.775	3.78
Hybrid (0.6 / 0.5)	4	2	0.720	2.78

Target videos. Pipeline A: the target in MOT16-01 has ID 11 in frames 1 to 361, with no missing frame inside the track. The video is in the repository of Pipeline A (`https://github.com/Rehan-rahim/mot16-pedestrian-tracking_project/tree/main/outputs/videos`). Pipeline B: the target in MOT16-14 has ID 215. The video shows the full sequence (750 frames), and the target is highlighted in about frames 544 to 724. In the six sampled frames of Figure 5, the box is on the same man in a dark jacket as he crosses the road toward the camera; we did not check every frame. The videos of Pipeline B (MOT16-14 target, MOT16-03, and the MOT16-09 validation video) are in one folder: `https://drive.google.com/drive/folders/19Z_0Swv9Cj_GKvjMxshVTRSGltaQ7dk9`. In MOT16-03, the track of the person with ID 166 is visible in frames 352 to 735 and then changes its ID. We chose the targets as clean examples, so the videos show favorable cases: Table 10 shows that most people are not tracked with one ID.



Figure 4: Target of Pipeline A (ID 11) at evenly spaced moments of MOT16-01.

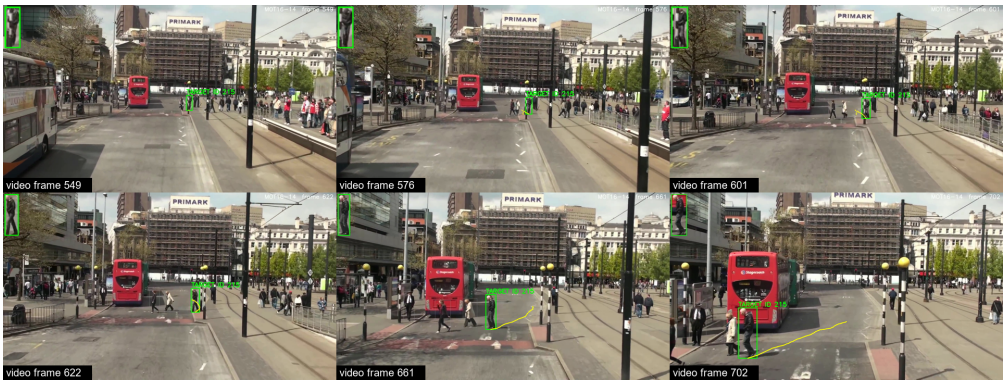


Figure 5: Target of Pipeline B (ID 215) in MOT16-14: six sampled frames of the target video (zoom inset at top left, yellow line: recent path).

4.5 Speed

Table 11 gives the speeds. They were measured on different GPUs, so they are not comparable. In Pipeline A, the detector uses 89% of the time.

Table 11: Speed of the pipelines (different GPUs; not comparable).

Stage	ms per frame	fps
A: Faster R-CNN detection (Colab T4)	104.17	9.6
A: Siamese ReID	10.41	–
A: tracker	2.39	–
A: whole pipeline (Colab T4)	116.97	8.55
B: detection and tracking (GPU not recorded)	–	about 29

The ReID embedding and the repair step of Pipeline B are offline and add time that this table does not include.

4.6 Limitations

- We use one validation sequence (MOT16-09), and it has large pedestrians (all at least 120 px).
- Both pipelines chose thresholds or checkpoints on MOT16-09, and the hybrid result is the best of four settings, so all scores are optimistic.
- The scores of YOLO and Faster R-CNN have different scales, so equal thresholds do not mean equal behavior.
- The test sequences have no public labels, so target videos are qualitative, and we chose the targets ourselves.
- The speeds come from different GPUs.
- The ReID comparison mixes network size and training method, and the checkpoint of Pipeline B was chosen on MOT16-09.
- Causes that we call likely (extra ID switches of the repair, undetected static people, extra false positives after gap filling) were not tested.

- Our reproduction of Pipeline A has the same detections (1,073 false positives; 1,215 against 1,214 misses) but an IDF1 of 62.38 against 60.35 for the published tracks; we did not investigate the cause.
- The tracker of Pipeline B is a library, and its appearance is not used inside it.

5 Conclusion

We built two offline tracking pipelines for MOT16 and compared them with the same evaluation code on MOT16-09. The detectors cover a similar share of pedestrians, Pipeline B outputs fewer false boxes, Pipeline A keeps identities more stable than BoT-SORT alone, and the repair gives Pipeline B the highest HOTA (55.30) and IDF1 (68.04) with more ID switches (56), but no run tracks most people with one ID. Next, we would test a second validation sequence, train the detectors with static-person labels, and apply the repair to the tracker of Pipeline A.

References

[1] Nir Aharon, Roy Orfaig, and Ben-Zion Bobrovsky. BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.

[2] Keni Bernardin and Rainer Stiefelhagen. Evaluating multiple object tracking performance: The CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing*, 2008.

[3] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *ICIP*, 2016.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[5] Alexander Hermans, Lucas Beyer, and Bastian Leibe. In defense of the triplet loss for person re-identification. *arXiv:1703.07737*, 2017.

[6] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLO. Software, `https://github.com/ultralytics/ultralytics`, 2026. Software, accessed 2026; version number not stated.

[7] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[8] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2:83–97, 1955.

[9] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017.

[10] Jonathon Luiten, Aljosa Osep, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe. HOTA: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021. doi: 10.1007/s11263-020-01375-2.

[11] Hao Luo, Youzhi Gu, Xingyu Liao, Shenqi Lai, and Wei Jiang. Bag of tricks and a strong baseline for deep person re-identification. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.

[12] Anton Milan, Laura Leal-Taixé, Ian Reid, Stefan Roth, and Konrad Schindler. MOT16: A benchmark for multi-object tracking. *arXiv:1603.00831*, 2016.

[13] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[14] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[15] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[16] Ergys Ristani, Francesco Solera, Roger Zou, Rita Cucchiara, and Carlo Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. In *ECCV Workshops*, 2016.

[17] Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato, and Lior Wolf. DeepFace: Closing the gap to human-level performance in face verification. In *CVPR*, 2014.

[18] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *ICIP*, 2017.

[19] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *ECCV*, 2022.

[20] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *AAAI*, 2020.